

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
Facultad de Ciencias de la Computación
Carrera de Ingeniería de Software

TA-IND-04 — Informe Técnico Individual

Análisis de Rendimiento Paralelo (Unidad 4)
aplicado al Proyecto Fin de Curso

Asignatura: Aplicaciones Distribuidas (ISR-701)
Unidad 4 — Cómputo Paralelo y Distribuido
Período académico: 2026–2027 PPA

Estudiante:	Cristhian Daniel Pacheco Cárdenas
Equipo de PE-U4:	Equipo A — Cristhian Daniel Pacheco Cárdenas, Robinson Rodrigo Cando Moreno, Ernesto Gregory Luna Mora
PFC de referencia:	ACC — Sistema de Gestión de Soporte Técnico ISP
Transformación foco:	T3 (Join, <i>tickets</i> $\bowtie$ <i>agentes</i>)
Repositorio individual:	https://github.com/CristhianP03/ta-ind-04-pacheco
Fecha:	8 de agosto de 2026

Docente: Gleiston C. Guerrero-Ulloa, M.Sc.

1. Introducción y contexto

Este informe analiza, de forma individual, los resultados de speedup obtenidos en la práctica grupal GA-SUM-05/PE-U4, en la que el equipo A implementó y midió cinco transformaciones (T1–T5) sobre un dataset sintético de 600 000 tickets de soporte técnico ISP, comparando pandas (línea base de una sola máquina) contra PySpark 3.5.3 en Google Colab, con escalado controlado de T3 (unión *tickets*×*agentes*) en $N = 1, 2, 4$ executors. El objetivo de este documento no es repetir ese experimento grupal, sino someter sus resultados a un análisis cuantitativo propio frente al modelo de Amdahl, diagnosticar la fracción no escalable observada, y fundamentar — con datos propios de PE-U4 y la arquitectura real del Proyecto Fin de Curso — una propuesta concreta de integración del procesamiento distribuido en dicho proyecto.

Los datos base se tomaron del repositorio <https://github.com/CristhianP03/pe-u4-spark-equipoA>, commit `aac987686e1d8dd0559b3ea6ed07af3335570ced`, archivo `resultados/tiempos_resumen.csv`, verificado línea por línea contra las mediciones crudas del mismo repositorio antes de su uso en este documento. Se descartó explícitamente el uso de cualquier tabla o cálculo del informe grupal (`docs/PE_U4_Informe.pdf`) más allá de los datos crudos de tiempos: el ajuste de Amdahl, la métrica de Karp-Flatt y el umbral de rentabilidad presentados aquí emplean una metodología propia, distinta de la reportada por el equipo, como se declara explícitamente en la Sección 9.

El dominio de referencia es **ACC — Sistema de Gestión de Soporte Técnico ISP**, un sistema de microservicios (Spring Boot, Node.js y FastAPI) con persistencia sobre un clúster CockroachDB de 3 nodos, en cuya Entrega 3 ya existe un *pipeline* de Spark propio para detección de reincidencia de clientes y *clustering* de tickets — pipeline cuya integración formal, periodicidad, dimensionamiento y consumidor de salida son precisamente el objeto de la Sección 5 de este informe.

2. Análisis propio de los resultados de speedup

2.1. Marco matemático

La ley de Amdahl relaciona el speedup $S(N)$ obtenido con N unidades de procesamiento y la fracción paralelizable p del cómputo:

$$S(N) = \frac{1}{(1-p) + \frac{p}{N}}, \quad S_{\text{máx}} = \lim_{N \rightarrow \infty} S(N) = \frac{1}{1-p}. \quad (1)$$

2.2. Datos base

Los tiempos de ejecución se tomaron del commit `aac987686e1d8dd0559b3ea6ed07af3335570ced` del repositorio de PE-U4 del equipo A, archivo `resultados/tiempos_resumen.csv`, generado sobre Google Colab con PySpark 3.5.3. Cada valor corresponde a la mediana de 5 repeticiones cronometradas, tras descartar una ejecución de calentamiento por combinación (transformación, motor).

2.3. Análisis agregado de las cinco transformaciones

Las cinco transformaciones muestran speedups a $N = 4$ entre 1.467 (T4, columna derivada) y 2.558 (T5, top- N). T5 obtiene el mayor beneficio porque su costo dominante es una agregación con muy pocas filas de salida, que Spark resuelve con un *shuffle* reducido tras la fase de agregación parcial por partición. T4, en cambio, aplica transformaciones fila por fila sin reordenar datos entre particiones; pandas ya ejecuta esa operación de forma vectorizada y eficiente en memoria, de modo que el margen de mejora al distribuirla es menor.

Cuadro 1: Tiempos medianos y speedup medido por transformación.

Transf.	Motor	N	T (s)	$S(N)$
T1	pandas	—	9,437	
T1	Spark	4	6,210	1,520
T2	pandas	—	8,345	
T2	Spark	4	4,363	1,913
T3	pandas	—	10,119	
T3	Spark	1	7,891	1,282
T3	Spark	2	7,059	1,433
T3	Spark	4	7,087	1,428
T4	pandas	—	12,296	
T4	Spark	4	8,381	1,467
T5	pandas	—	6,269	
T5	Spark	4	2,450	2,558

2.4. Desarrollo matemático completo para la transformación foco (T3)

Para T3 se dispone de mediciones con $N = 1, 2, 4$ executors, lo que permite ajustar la fracción paralelizable p de la ecuación (1) por mínimos cuadrados. Se excluye deliberadamente el punto $N = 1$ del ajuste: la ley de Amdahl impone $S(1) = 1$ por construcción (sin paralelismo no hay aceleración posible), mientras que el valor medido es $S(1) = 1,282$; esa diferencia no proviene de paralelismo — imposible con un único executor — sino de que Spark y pandas son motores de ejecución distintos, y el optimizador de consultas de Spark ya introduce ganancias de motor independientes del número de executors. Incluir ese punto en el ajuste distorsionaría la estimación de p sin aportar información sobre escalabilidad real.

Minimizando $\sum_{N \in \{2,4\}} (S_{\text{med}}(N) - S(N;p))^2$ se obtiene $\hat{p} = 0,4445$, con lo que la ecuación (1) predice $S_{\text{teo}}(2) = 1,286$, $S_{\text{teo}}(4) = 1,500$ y un techo asintótico $S_{\text{máx}} = 1/(1 - \hat{p}) = 1,800$.

Cuadro 2: Speedup medido frente al predicho por la ley de Amdahl ($\hat{p} = 0,4445$).

N	S_{med}	S_{teo}	Δ (%)
2	1,433	1,286	11,49
4	1,428	1,500	-4,82

2.5. Respuesta a la pregunta 1 de la guía

Los resultados no coinciden con la predicción de Amdahl de forma uniforme: a $N = 2$ el speedup medido *supera* al teórico en un 11,49 %, mientras que a $N = 4$ queda *por debajo* en un 4,82 % (Tabla 2, Figura 1). Esta inversión de signo es, en sí misma, el hallazgo relevante. Con dos executors, la ganancia adicional de paralelismo todavía compensa la sobrecarga de coordinación, y el optimizador de Spark aporta una mejora de motor no capturada por el modelo de Amdahl. Con cuatro executors, en cambio, el costo de coordinar más particiones — reparto de trabajo del *join*, transferencia de datos entre executors y planificación de tareas — empieza a superar la ganancia marginal de paralelismo, algo que Amdahl no puede anticipar porque asume implícitamente que la única variable es N y que el costo de coordinación es despreciable **amdahl1967**. Hill y Marty señalan precisamente que este tipo de rendimientos decrecientes se agrava cuanto mayor es el número de unidades coordinadas, incluso

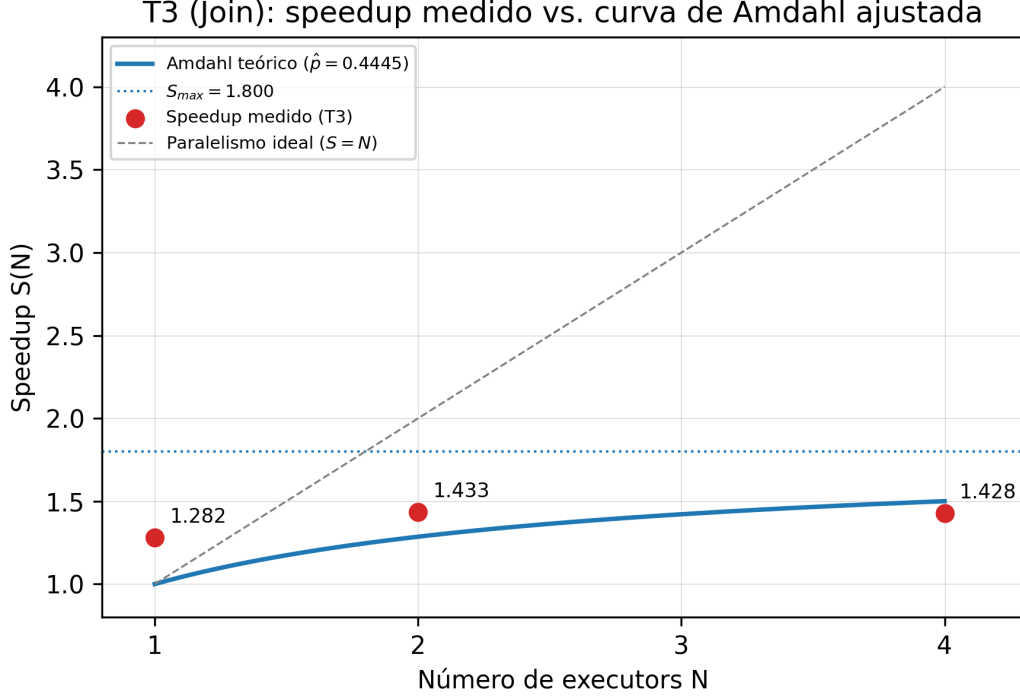


Figura 1: Speedup medido de T3 (unión pandas/Spark) frente a la curva teórica de Amdahl ajustada por mínimos cuadrados ($\hat{p} = 0,4445$). La brecha entre la curva de paralelismo ideal ($S = N$) y la curva teórica refleja el límite estructural impuesto por la fracción no paralelizable; la brecha entre la curva teórica y los puntos medidos refleja la sobrecarga real del motor, que se cuantifica en la Sección 4 mediante la métrica de Karp-Flatt.

en arquitecturas de propósito general **hillmarty2008**. La Sección 4 retoma esta misma inversión con la métrica de Karp-Flatt, que confirma cuantitativamente que la causa es la sobrecarga creciente del motor y no un límite algorítmico fijo.

3. Diagnóstico de la fracción no escalable

3.1. Métrica de Karp-Flatt

La fracción serial experimental e , determinada a partir del speedup observado S con N unidades, se define como:

$$e = \frac{\frac{1}{S} - \frac{1}{N}}{1 - \frac{1}{N}}, \quad N > 1. \quad (2)$$

Sustituyendo en la ecuación (2) los valores medidos de T3 (Tabla 2):

$$e(2) = \frac{\frac{1}{1,43346} - \frac{1}{2}}{1 - \frac{1}{2}} = \frac{0,69761 - 0,5}{0,5} = 0,39522 \quad (3)$$

$$e(4) = \frac{\frac{1}{1,42772} - \frac{1}{4}}{1 - \frac{1}{4}} = \frac{0,70042 - 0,25}{0,75} = 0,60055 \quad (4)$$

La fracción serial experimental **crece un 51.95 %** al pasar de $N = 2$ a $N = 4$ ($0,395 \rightarrow 0,601$). Según el criterio de interpretación de Karp y Flatt **karpflatt1990**, un valor de e que se mantiene aproximadamente constante señala un límite algorítmico fijo, mientras que un valor creciente indica sobrecarga adicional del motor de ejecución al aumentar N . El fuerte incremento observado descarta la primera hipótesis: la pérdida de escalabilidad de T3 entre $N = 2$ y $N = 4$ no es atribuible a la fracción secuencial intrínseca del algoritmo de *join*, sino a costos de coordinación que crecen con el número de executors.

3.2. Eficiencia del paralelismo

La eficiencia mide qué fracción del beneficio ideal se aprovecha realmente por cada unidad de procesamiento adicional:

$$E(N) = \frac{S(N)}{N}. \quad (5)$$

Con los valores medidos de T3, $E(1) = 1,282$, $E(2) = 0,717$ y $E(4) = 0,357$ (ecuación (5)): la eficiencia cae de forma pronunciada y monótona incluso donde el speedup absoluto todavía crece (de $N = 1$ a $N = 2$), lo que confirma visualmente el mismo fenómeno que Karp-Flatt cuantifica — cada executor adicional aporta una fracción decreciente de beneficio real.

3.3. Atribución a mecanismos concretos del motor

La captura de la Spark UI para T3 con $N = 4$ (evidencia/spark_ui_t3.png del repositorio de PE-U4, *query* SQL de duración total 12s) permite identificar cuatro mecanismos concretos que explican el crecimiento de e :

1. **Colapso adaptativo de particiones (AQE).** El nodo AQEShuffleRead reporta `number of coalesced partitions`: 1 en ambos lados del *join*. Aunque la sesión de Spark se configuró con 4 executors, el motor de ejecución adaptativa reunió el *shuffle* completo en una única partición por el volumen reducido de datos (180 003 registros tras el filtrado), de modo que buena parte de la etapa final del *pipeline* corrió efectivamente en un solo executor, independientemente de N .
2. **Costo de *exchange* (shuffle) del *join*.** Los nodos Exchange muestran 180 003 registros escritos en *shuffle* en el lado de `tickets`, con un tiempo de escritura de hasta 604ms en el peor *task*. Al usar `sort-merge join` con difusión desactivada deliberadamente, ambas tablas — incluida `agentes`, de solo 300 filas — se redistribuyen por red en lugar de aprovechar una unión por difusión.
3. **Ordenamiento previo al *merge*.** Los nodos WholeStageCodegen de tipo Sort muestran un tiempo de ordenamiento de hasta 594ms antes de ejecutar el `SortMergeJoin`, etapa inherente a la estrategia de *join* cuyo costo no se reduce por agregar executors.

4. **Planificación del DAG y evaluación perezosa.** Spark construye el plan de ejecución completo antes de materializarlo, y en este caso generó 4 *jobs* sucesivos debido a la re-planificación adaptativa tras recolectar estadísticas de las etapas iniciales **zaharia2016**. Esa coordinación adicional es un costo fijo que no escala con los datos ni se beneficia de más executors.

En conjunto, estos cuatro mecanismos — y en particular el colapso de particiones por AQE — muestran que la sobrecarga de T3 no es un fenómeno difuso atribuible de forma genérica al motor, sino un efecto localizable y verificable en la propia Spark UI de la ejecución medida.

4. Propuesta de integración al PFC

4.1. Caso de uso

El PFC ACC (Sistema de Gestión de Soporte Técnico ISP) ya incorpora, desde su Entrega 3, un *pipeline* de Apache Spark orientado a dos análisis sobre el histórico de tickets almacenado en `ticket_db` (CockroachDB, clúster de 3 nodos): (i) detección de **reincidencia de cliente**, mediante una función de ventana (`Window.partitionBy(client_id).orderBy("timestamp")`) que calcula, para cada cliente, el tiempo transcurrido desde su incidencia anterior y marca reincidencia cuando esa diferencia es menor a 30 días; y (ii) **clustering** de tickets con K-Means ($k = 5$), sobre *features* de TF-IDF de la descripción textual combinadas con la zona operativa codificada. Para evitar ambigüedad con la nomenclatura de PE-U4, estas transformaciones se referencian aquí como $T_{\text{reincidencia}}$, T_{flag30d} y $T_{\text{clustering}}$, distintas de T1–T5 de la práctica grupal.

4.2. Periodicidad y volumen (declarados como decisión propia)

El pipeline se ejecutó, hasta ahora, una sola vez de forma manual desde notebook; no existe una periodicidad definida en el PFC. Se propone, como decisión de diseño de este informe, una **ejecución batch mensual**, alineada con el cierre natural de los reportes gerenciales de soporte técnico. Para dimensionar esa ejecución se declara el siguiente supuesto, explícitamente no derivado de datos medidos del PFC: un ISP mediano opera con un volumen de **aproximadamente 1 500 tickets mensuales** (rango de sensibilidad 500–2 000, según estacionalidad de incidencias de red). Este supuesto alimenta el cálculo del umbral de rentabilidad de la Sección 6.

4.3. Salida producida y decisión de negocio soportada

La ejecución mensual del pipeline produce, por cliente: la bandera `is_recurring_30d` y el `cluster_id` asignado a cada ticket, agregados en dos tablas de salida: (a) conteo de clientes reincidentes por zona y categoría, y (b) distribución de tickets por clúster y zona. Esta salida soporta una decisión operativa concreta: priorizar la atención de campo hacia zonas con alta concentración de clientes reincidentes, en vez de tratar cada ticket como un evento aislado — un uso análogo, en espíritu, al enrutamiento automatizado de tickets hacia el grupo experto correcto que describen Xu et al. **xu2018** para sistemas de gestión de TI, aunque aquí el objetivo no es el enrutamiento en sí sino la identificación proactiva de patrones de reincidencia antes de que se conviertan en escaladas de SLA.

4.4. Punto de inserción en la arquitectura en capas

La arquitectura de ACC es de microservicios con base de datos por servicio: **auth-service**, **ticket-service** y **report-service** (Spring Boot/Java) sobre CockroachDB; **notification-service** (Node.js) y **ai-service** (FastAPI/Python) sobre MongoDB; comunicación asíncrona entre servicios

vía Kafka. El pipeline de Spark se inserta como un **proceso batch independiente**, fuera del flujo síncrono de creación/clasificación de tickets: lee mensualmente por JDBC directamente de `ticket_db`, sin pasar por Kafka ni por el `ai-service`, y escribe sus tablas de salida en un esquema separado dentro del mismo clúster CockroachDB. `report-service` — que ya expone lectura des-
acoplada bajo un patrón CQRS — añade un **endpoint nuevo** que consulta esas tablas de salida y las expone en un panel para el rol Administrador del frontend (Figura 2).

Integración batch de Spark en ACC — elaboración propia

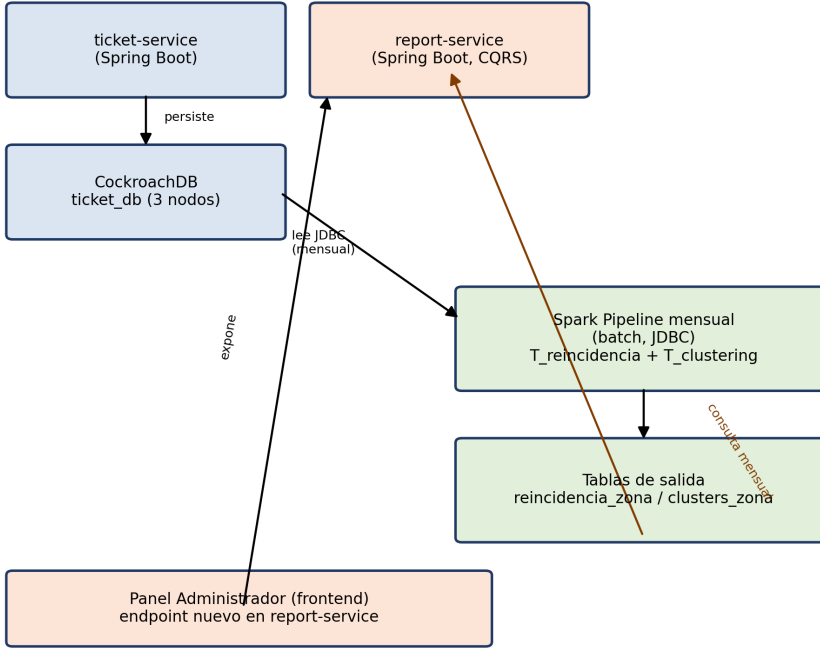


Figura 2: Punto de inserción del pipeline de Spark en la arquitectura de microservicios de ACC. Elaboración propia.

Es deliberado que la integración sea por **lotes y no en flujo**, pese a que el resto del sistema es orientado a eventos: la detección de reincidencia requiere una ventana histórica de 30 días por cliente, una agregación que no tiene sentido recalcularlo evento por evento en tiempo real, y que además no puede alimentar de vuelta al `ai-service` sin rediseñar su contrato de clasificación síncrona — una limitación de integración que se reconoce explícitamente aquí y se retoma como limitación en la Sección 7.

5. Dimensionamiento y viabilidad

El umbral de rentabilidad modela el tiempo secuencial como $T_{\text{sec}}(n) = \alpha n$ y el distribuido como $T_{\text{dist}}(n) = \beta + \gamma n/N$, siendo ventajoso el motor distribuido cuando $T_{\text{dist}}(n) < T_{\text{sec}}(n)$:

$$n^* = \frac{\beta}{\alpha - \frac{\gamma}{N}}, \quad \text{válido si } \alpha > \frac{\gamma}{N}. \quad (6)$$

Método de ajuste declarado: $\hat{\alpha}$ se estima como T_{pandas}/n sobre el punto medido de T3 ($n = 600\,000$); $\hat{\beta}$ y $\hat{\gamma}$ se obtienen por regresión lineal de T_{dist} contra $1/N$ sobre los tres puntos medidos de T3 ($N = 1, 2, 4$), ya que $T_{\text{dist}} = \beta + (\gamma n) \cdot (1/N)$ es lineal en $1/N$ para n fijo. El resultado es $\hat{\alpha} = 1,6865 \times 10^{-5}$ s/registro, $\hat{\beta} = 6,6711$ s, $\hat{\gamma} = 1,9280 \times 10^{-6}$ s/registro. Sustituyendo en la ecuación (6) con $N = 4$:

$$n^* = \frac{6,6711}{1,6865 \times 10^{-5} - \frac{1,9280 \times 10^{-6}}{4}} \approx 407,206 \text{ registros.} \quad (7)$$

Con el volumen mensual declarado en la Sección 5 ($\sim 1\,500$ tickets/mes), se tardarían aproximadamente 271 meses (~ 22 años) en acumular ese volumen en una sola ejecución. **Recomendación:** Spark no se justifica para el batch mensual de detección de reincidencia bajo este modelo; se reserva para el histórico acumulado completo o para un reentrenamiento anual del clustering, mientras que la detección mensual de reincidencia (ventana corta, 30 días) se resuelve por debajo del umbral con una consulta SQL directa sobre CockroachDB (ver alternativa evaluada en la Sección 7.2). El modelo de servicio en la nube recomendado para las ejecuciones que sí superen n^* es un clúster gestionado de sesión efímera (por ejemplo, Databricks Job Clusters o AWS EMR Serverless), justificado por el modelo de costos *pay-as-you-go* que describen Armbrust et al. **armbrust2010**: al ejecutarse solo mensualmente o anualmente, mantener un clúster persistente no se justifica económicamente frente a la elasticidad de aprovisionamiento bajo demanda. El principal riesgo identificado es que $\hat{\beta}$ se estimó sobre una sesión de Colab, no representativa del costo de arranque real de un clúster gestionado en producción (ver limitación declarada en la Sección 7.4).

6. Postura crítica y alternativas descartadas

6.1. ¿Es Spark la opción correcta para ACC?

La respuesta es condicional, no binaria, y se sostiene directamente en los resultados de las secciones anteriores. A nivel de motor, Spark demostró ser técnicamente superior a pandas en las cinco transformaciones medidas en PE-U4 (Tabla 1), y el análisis de Karp-Flatt (Sección 4) permitió diagnosticar con precisión que su límite de escalado en T3 proviene de sobrecarga de coordinación — parcialmente explicada por el colapso adaptativo de particiones observado en la Spark UI — y no de una limitación algorítmica del *join*. En ese plano, Spark es una elección sólida y validada empíricamente con datos propios, no una moda tecnológica adoptada sin evidencia.

Sin embargo, el cálculo del umbral de rentabilidad (Sección 6) arroja $n^* \approx 407,206$ registros, mientras que el volumen mensual supuesto para ACC ($\sim 1\,500$ tickets/mes) tardaría más de 20 años en alcanzarlo. **Para el caso de uso mensual descrito en la Sección 5, Spark no está justificado hoy:** el propio modelo matemático que motiva su adopción, aplicado con honestidad a los números reales del PFC, recomienda lo contrario a la intuición inicial del equipo. La postura de este informe es, por tanto, que Spark debe reservarse para cargas de trabajo que sí crucen ese umbral — el histórico acumulado completo, o un reentrenamiento anual del clustering — y no para la detección mensual de reincidencia, que se resuelve de forma más simple y con menor costo operativo sin un motor distribuido.

6.2. Alternativa 1 descartada: agregación en SQL directamente sobre CockroachDB

CockroachDB soporta funciones de ventana SQL nativas, equivalentes a la lógica de `Window.partitionBy(client_id).orderBy("timestamp")` que hoy implementa $T_{\text{reincidencia}}$ en Spark. Para

el volumen mensual de ACC, muy por debajo de n^* , una consulta SQL directa sobre `ticket_db` evitaría por completo el costo fijo β de arrancar una sesión Spark, sin sacrificar corrección. Se descarta como reemplazo definitivo — no como solución mensual, que sí se recomienda — porque no escala si ACC crece varios órdenes de magnitud o si se agrega el *clustering* sobre texto, que SQL no puede expresar sin extensiones externas; conviene mantenerla exclusivamente para el escenario de bajo volumen ya identificado.

6.3. Alternativa 2 descartada: Dask en lugar de Spark

Dask ofrece paralelismo de datos en Python puro, sin la sobrecarga de serialización JVM $\leftrightarrow$ Python (Py4J) que Spark introduce incluso en modo local, y encajaría de forma más homogénea con el stack Python ya presente en `ai-service` (FastAPI). Se descarta para ACC por dos razones técnicas concretas: (i) el equipo ya validó empíricamente su dominio de Spark en PE-U4, incluida la verificación de equivalencia numérica `pandas` $\leftrightarrow$ Spark, mientras que migrar a Dask reiniciaría ese trabajo de validación sin garantía de mejor resultado a este volumen; y (ii) el conector JDBC de Spark hacia CockroachDB es más maduro y mejor documentado que el soporte equivalente en el ecosistema de Dask, lo que reduce el riesgo de integración en el punto de inserción ya definido en la Sección 5.

6.4. Limitaciones reconocidas de esta propuesta

Dos limitaciones deben declararse explícitamente. Primero, el *clustering* de $T_{\text{clustering}}$ concentra el 81,3 % de los registros en un único clúster, un artefacto de la baja variabilidad léxica de las descripciones generadas por plantilla en el dataset sintético; su valor de negocio real solo podrá evaluarse con descripciones de clientes reales, de mayor diversidad textual, en producción. Segundo, los parámetros $\hat{\beta}$ y $\hat{\gamma}$ del umbral de rentabilidad se estimaron sobre una sesión de Spark en Google Colab, cuyo costo fijo de arranque puede no ser representativo de un clúster gestionado en producción (por ejemplo, Databricks o EMR); por tanto, $n^* = 407,206$ debe leerse como una estimación de orden de magnitud, no como un umbral operativo exacto para un despliegue real de ACC **hillmarty2008**.

7. Conclusiones

1. El ajuste de la ley de Amdahl sobre T3 ($\hat{p} = 0,4445$, Sección 3) predice una curva monótonamente creciente, pero los datos medidos muestran una inversión de signo entre $N = 2$ y $N = 4$ ($\Delta = +11,49\%$ frente a $\Delta = -4,82\%$, Tabla 2): el modelo clásico, que asume sobrecarga nula, es insuficiente por sí solo para explicar el comportamiento real de un motor distribuido y debe complementarse con un diagnóstico de sobrecarga como el de Karp-Flatt.
2. La fracción serial experimental de T3 crece un 51.95 % entre $N = 2$ y $N = 4$ ($e : 0,395 \rightarrow 0,601$, Sección 4), y la evidencia directa de la Spark UI — en particular el colapso adaptativo de particiones a una sola partición — confirma que la causa es sobrecarga de coordinación del motor, no un límite algorítmico del *join*; esto demuestra que atribuir la pérdida de escalabilidad a “el motor” de forma genérica es insuficiente cuando existe evidencia verificable que apunta a un mecanismo específico.
3. El umbral de rentabilidad calculado para el PFC ACC ($n^* = 407,206$ registros, Sección 6) es muy superior al volumen mensual realista supuesto ($\sim 1\,500$ tickets/mes, Sección 5): la decisión técnicamente honesta, sostenida en el propio modelo matemático exigido por esta actividad, es no adoptar Spark para el procesamiento mensual de reincidencia, reservándolo para el histórico

acumulado completo una conclusión que contradice la intuición inicial de “más datos, siempre Spark” y que solo emerge de cuantificar el umbral con datos propios en vez de asumirlo.

Declaración de uso de inteligencia artificial generativa

Se utilizó Claude (Anthropic) como herramienta de apoyo en las siguientes tareas, todas revisadas y validadas por el autor antes de su inclusión: (i) verificación técnica del repositorio de PE-U4 clonación, comprobación de accesibilidad anónima, cotejo línea por línea de tiempos crudos.csv/tiempos resumen.csv contra el commit declarado, y revisión de **references_U4.bib** para confirmar qué referencias bibliográficas eran de aporte propio; (ii) cálculos numéricos ajuste por mínimos cuadrados de la fracción paralelizable $\hat{p}$, sustitución numérica de la métrica de Karp-Flatt, y regresión lineal para estimar $\hat{\alpha}, \hat{\beta}, \hat{\gamma}$ del umbral de rentabilidad, ejecutados como código verificable, no estimados a mano; (iii) generación de las figuras propias (**fig_speedup.png** y **fig_arquitectura_integracion.png**) mediante scripts de Python, incluida su corrección iterativa tras detectar errores tipográficos y de dirección lógica en las flechas; y (iv) redacción asistida de las Secciones 3 a 8, sobre la base de datos, decisiones arquitectónicas y respuestas de dominio aportadas íntegramente por el autor.

Se declara explícitamente una divergencia metodológica identificada durante el proceso: el ajuste de $\hat{p} = 0,4445$ obtenido en este informe individual, sobre $N = 2, 4$, difiere del valor $p = 0,173423$ reportado en el informe grupal de PE-U4, que empleó una metodología de ajuste distinta. Esta diferencia se declara para dejar constancia de que el análisis matemático de este documento es independiente del grupal, no una reproducción de sus resultados.